

VIOLIN VIBRATO TONE SYNTHESIS

TIME-SCALE MODIFICATION AND ADDITIVE SYNTHESIS

By
Yizhen Ding

Senior Thesis in Electrical Engineering
University of Illinois at Urbana-Champaign
Advisor: James Beauchamp

May 2019

© Copyright by Yizhen Ding, 2019

Abstract

This thesis reports on the development of a program to synthesize music based on real violin vibrato tones. General time-scale modification methods cannot deal with vibrato tones because they are not able to preserve vibrato rates while modifying tone duration. Vibrato parameterization, time-scale modification, and additive synthesis were used to overcome this limitation.

The analysis and synthesis process can be divided into several steps: First, to analyze a single tone we extract the frequency and amplitude information for each harmonic and each time frame using either the harmonic frequency McAulay-Quatieri (MQHF) method or the phase-vocoder (PV) analysis method. Then we identify the attack and decay portions of the resulting time-varying harmonic envelopes. Next for the time region between the attack and decay we parameterize the analysis file using three time-varying parameters: vibrato rate, vibrato amplitude, and mean value. These are applied to both the amplitude and the frequency of each harmonic. For the duration of each synthesized tone, the synthesized frequency and amplitude data are calculated by stretching or compressing the magnitude and frequency parameters while preserving the modulation rate and the attack, and decay. Finally, sounds are converted to the time domain by adding the sinusoidal components together. To make music, a score file is written in the Notepro alphanumeric language, which resembles conventional music notation. This translates into a Music 4C score file that includes the start time, duration, pitch, amplitude, vibrato rate and analysis file for each note. By using all of the methods described above, a Music 4C program was made to synthesize a piece of violin music according to a given score file.

Subject Keywords: music signal processing; vibrato synthesis; time-scaling modification; additive synthesis;

Acknowledgments

I would first like to thank my thesis advisor Prof. James Beauchamp for the continuous support of my research, for his patience, motivation, and immense knowledge.

I would also like to acknowledge Yang Shi of the electrical computer engineering department at University of California, Los Angeles, for developing the first version of this program.

Contents

1. Introduction.....	1
1.1 <i>M4C</i> Package	1
1.1.1 <i>M4C</i> instruments.....	1
1.1.2 <i>M4C</i> score files	2
1.2 <i>Notepro</i> Package	3
1.2.1 Pitch notation	3
1.2.2 Duration notation	3
2. Previous Work.....	4
3. Description of Research Results	6
3.1 Harmonic Analysis.....	7
3.2 Vibrato Parameterization.....	8
3.2.1 Frequency and amplitude spectrum	8
3.2.2 Attack and decay	9
3.2.3 Parametric model	10
3.2.4 Estimated vibrato rate	11
3.2.5 Heterodyne method	12
3.3 Time-scale Modifications.....	13
3.3.1 Most cases	13
3.3.2 A Special case	14
3.4 Additive Synthesis	15
4. Conclusion	16
References.....	17

1. Introduction

Vibrato is a musical effect consisting of a periodic change of pitch. In Western music a vibrato effect is commonly employed in singing and with bowed-string performance in order to add warmth or interest to notes. ^[1] In this research, we mainly focused on the bowed-string instrument, especially the violin.

Typically, the vibrato rate for a bowed string is between 5 and 7 Hz. The vibrato amplitude (depth) for a string instrument is generally less than 2%. Currently, there is no violin synthesis program that is able to synthesize a vibrato tone from a given violin sound sample. General time-scale modification methods, which are often used to synthesize sound from a given sample, are not able to preserve the vibrato rate while modifying the duration. Due to the limitation of general time-scale modification methods, we designed a parametric model for vibrato tones. Synthesis of vibrato tones can be performed by preserving parameters of the sample tone.

Synthesis was implemented by a Music 4C (M4C) program as an instrument named *vb*. The program requires at least one sample analysis tone and one score file (*.sc). Synthetic violin tones are generated by a novel synthesis program inside M4C that reads and processes harmonic analysis files. Score files are first composed in the *Notepro* format (*.np) and then translated into numerical score files (*.sc) using a special version of the notepro program named *NoteproVB*.

To give more detail, the M4C program first reads harmonic analysis sound files and then calculates the vibrato parameters by using a parametric model. Next, by preserving some parameters, the program is able to perform time-scale modification according to the note durations specified in the score file. The result after time-scale modification is new harmonic spectrum information. With the additive synthesis method, the program is able to synthesize the desired vibrato tone based on the harmonic spectrum, pitch and dynamic specified by the score file. Eventually, the program generates a piece of music by repeating this process for many notes.

1.1 M4C Package

Music 4C (M4C) is a software music synthesis program developed by Scot Aurenz at the University of Illinois at Urbana-Champaign in 1985. It was written in C and is supported by the Unix (including Linux) environment. ^{[2][3]}

1.1.1 M4C instruments

As a music synthesis program, M4C can be used to synthesize sequences of sounds whose pitch, start time, duration, dynamic, and timbre are specified by a score file. There are many available instruments in the M4C package: glide tone, piano, plucked instrument, etc. Most of the instruments are synthesized by

self-defined models. It is also possible to synthesize any acoustic instrument by using the additive synthesis method (i.e., the *m4c.as* instrument) as long as an analysis file for the instrument is provided. However, *m4c.as* heretofore has only been applicable for non-vibrato tones. Therefore, a new M4C instrument designed to handle vibrato tones was necessary.

1.1.2 M4C score files

In order to synthesize a piece of music which has sequence of notes, an appropriate score file is necessary. The score file should have *.sc* as the extension. Here is a typical score file *canon.sc*.

```
I(Plv1, 0.000, 3.0) { 8.0700, 2048, 60}
I(Plv1, 0.833, 3.0) { 9.0200, 2048, 60}
I(Plv1, 1.667, 3.0) { 8.1000, 2048, 60}
I(Plv1, 2.917, 3.0) { 8.0900, 2048, 60}
I(Plv1, 3.333, 3.0) { 8.0700, 2048, 60}
I(Plv1, 3.750, 3.0) { 8.1000, 2048, 60}
I(Plv1, 4.167, 3.0) { 8.0900, 2048, 60}
I(Plv1, 4.583, 3.0) { 8.0700, 2048, 60}
I(Plv1, 5.000, 3.0) { 8.0600, 2048, 60}
I(Plv1, 5.417, 3.0) { 8.0900, 2048, 60}
I(Plv1, 5.833, 3.0) { 8.0200, 2048, 60}
I(Plv1, 6.667, 3.0) { 8.0700, 2048, 60}
.....
End
```

Typically, each note can be specified by one line of code. Each instance starts with *I*. In this example, *Plv1* specifies the instrument name or the synthesis method. The next two arguments are the start time and the duration in second. The three other arguments inside the braces are pitch, dynamic (amplitude) and attack. The pitch is given in octave-point pitch class units, one of the many possible pitch/frequency formats. The octave is defined by the number to the left of the decimal point (the standard octave plus 4). The number on the right of the decimal point gives the pitch-within-the-octave. For example, 9.06 means the current note has pitch F# in the fifth octave. Therefore, for each octave, the pitch ranges from .00 (C) to .11 (B). Increasing the pitch by .01 means the pitch is a half-step higher. The score file must end with an End statement.

The actual meaning and the number of parameters can vary based on the instrument name or synthesis method. Here is the format for each instance of the new vibrato instrument:

```
I(vb, start_time, duration) {analysis_file, pitch, amplitude_scale, vibrato_rate, right_channel}
```

Here is an example:

```
I(vb, 0.000, 1.650) {"pvn400.2.mq.an" 9.06 0.6 5 0 }
```

1.2 Notepro Package

Writing a score file directly can be extremely taxing. Therefore, there is a program, *Notepro*^[4], which can convert a music-notation-like file, with an suffix of .np, to a score file required by *M4C* with a .sc suffix. *Notepro* is a music transcription tool for computer music programs. It translates alpha-numeric renditions of conventional music notation scores into numeric scores suitable for synthesis – except that some parameter information specific to the instrument must be entered in the statements, unless a specific version of notepro (e.g., notepro.vb) is compiled.

Notepro files contain instrument names, tempo, dynamic and pitch of a series of notes. Here is part of the *Notepro* file of Mendelssohn’s violin concerto in E minor:

```
'filename' fff T170
```

```
| [ ^ B. B/ ] | [ ^ Bo G E ^ ] | [ ^ E ^ o B G ^ ] | [ ^ F# E C E ] |
```

1.2.1 Pitch notation

In *Notepro*, we use *CDEFGAB* to denote seven basic notes in each octave. Pitch can be altered by half steps using the symbol “#” (sharp) or “b” (flat). By default, notes are at the fourth octave (e.g., A₄ corresponding to 440 Hz). The octave can be altered using “^” (octave up) or “v” (octave down). Repeating a pitch modifier doubles its effect: i.e., “^^” increases pitch by two octaves. Table 1 shows the seven basic notes in *Notepro* notation and their corresponding *M4C* notation:

Table 1: notation conversion between *Notepro* an *M4C*

<i>Notepro</i> notation	C	D	E	F	G	A	B
<i>M4C</i> notation	8.00	8.02	8.04	8.05	8.07	8.09	8.11

1.2.2 Duration notation

The default duration of each note is one beat in the current tempo. For example, in the *Notepro* file given above, the tempo is 170 beats per minute. Thus, the duration of a quarter note is 60/170 second. (Note that the occurrences of “|” are bar indicators; they are for appearance’s sake and have no effect on performance.) Repeating most modifiers multiplies their effect: e.g., “//” means that duration is quartered. However, as in conventional music notation, two dots does not square a single dots effect. Instead, two dots multiplies the duration by 1.75. Table 2 lists a few of the most frequently used duration modifiers:

Table 2: Duration modifiers in *Notepro*

Symbol	/	o	.
Meaning	Half duration	Double duration	Duration×1.5
Symbol	//	o o	..
Meaning	Quarter duration	4 times duration	Duration×1.75

2. Previous Work

Based on the goal of synthesizing violin vibrato tones using time-scale modification and additive synthesis, the author's research has mainly focused on control of vibrato and time-scale modification. There are some related researches in this field.

Robert C. Maher and James W. Beauchamp carried out an investigation of vocal vibrato for synthesis.^[5] This research mentioned the importance of vibrato as a character of sound quality. If convincing and natural synthesized sound is desired, vibrato must be treated with care. This is one of the most important reasons for designing a violin vibrato synthesis program. Almost all bowed-string instrument performances utilize vibrato, especially for long notes. Therefore, the vibrato feature is necessary for bowed-string instrument synthesis.

Maher and Beauchamp's paper also discussed the analysis method. Their research implied that the harmonic frequency McAulay-Quatieri (MQHF) frequency-tracking method is the best analysis method for vibrato tone analysis. Other analysis methods (such as the phase vocoder method) which use fixed-frequency filters to separate harmonics may generate unwanted "cross talk" between adjacent filters, since large harmonic frequency changes can oscillate between these filters. On the other hand, the MQHF method can track widely changing frequencies, so that it avoids this cross talk problem. Based on this research result, the MQHF method was adopted as the preferred analysis method for the violin vibrato spectral analysis.

Time-scale modification means stretching or compressing the duration of an audio sound.^[6] Ideal time-scale modification should preserve the pitch, amplitude and timbral quality of the original sound. There are many time-scale modification methods. Some of them operates in the time domain: e.g., the frame based Overlap-Add (OLA) method. Others focus on the frequency domain: e.g., the phase vocoder (PV) and sinusoidal spectral modeling methods.

The frame-based approach is the most straightforward approach. It first decomposes the original signal into many short frames. Each frame usually has duration about 50-100 milliseconds. Then the duration of the audio sound can be changed by changing the hopsize (time interval between two frames). Although this method is able to keep the pitch, it can lead to many problems. First, there can be discontinuities when relocating frames, resulting in a clicking effect in the output sound. Second, after changing the hopsize, the amplitude will also be modified when frames are added up. In order to overcome the discontinuity and reduce the unwanted amplitude fluctuation, the overlap-add (OLA) method multiplies by a window function on each frame before reconstructing the audio sound. However, this method is still not suitable enough for music synthesis because when applying OLA to harmonic signals, the output

sounds always have a warbling sound which is caused by a distortion of the sound's periodic pattern. Methods like synchronized overlap-add (SOLA) and waveform similarity overlap-add (WSOLA) have improved the basic OLA method. But although they can avoid unwanted warbling effects with non-vibrato sounds, these methods can not deal with vibrato tones. Vibrato patterns will also be stretched or compressed while modifying the duration.

The phase vocoder (PV) and sinusoidal spectral modeling are two frequency domain time-scale modification methods. The PV method is based on the short-time Fourier transform (STFT). It analyzes and records the audio signal's frequency and amplitude information at each frame. One way to interpret the PV is to consider it as a filter bank. The sinusoidal spectral modeling method is also known as the McAulay-Quatieri (MQ) method, which we have already discussed above. This method also records the frequency and amplitude information of each frame. Unlike PV, the MQ method models frequency components as tracks. Both PV and MQ analyze the audio sound by recording the frequency and amplitude. Thus, the synthesis from PV or MQ is just the addition of sine waves (additive synthesis). Since the frequency and amplitude values extracted using PV or MQ are continuous, there will not be any clicking for warbling effects. Time-scale modifications using either PV or MQ can then be accomplished by linear interpolation of the frequency and amplitude values. However, the vibrato rate will still be affected.

After investigating each method, we have found that the PV and MQ methods are able to return excellent sound quality when a vibrato tone's duration is preserved. But in order to make these two methods preserve the tone's vibrato rate over variable duration, a parametric model must be devised that incorporates vibrato features.

3. Description of Research Results

The entire procedure of this vibrato synthesis program can be divided into four parts: harmonic analysis, parameterization, time-scale modification and additive synthesis. Their relations are demonstrated by the block diagram in figure 1.

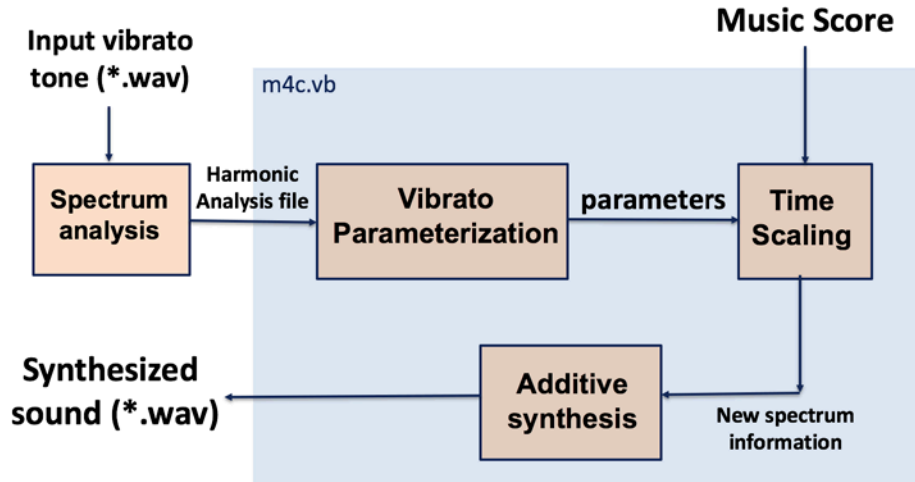


Figure 1: Block diagram of program procedure

To prepare for the synthesis, we need a collection of violin sound files. First, we need to do the spectrum analysis of these files, which creates a group of analysis files with the suffix “.an”. This harmonic analysis part uses a software package named *SN DAN*. The names of the analysis files, which are used to synthesize each of the notes in a piece, are included in an M4C score file.

With the score file and the analysis files, we can then start the synthesis. The M4C program *m4c.vb* is executed using the command “*gom4C*”; type “*vb*” for the method and then enter the score file’s name. Then the program will start the synthesis note by note. For each note, the program will first perform vibrato parameterization for the analysis file and extract parameters that represent the timbral quality. The duration of the original violin sound is scaled to the duration specified in the score while preserving some of the vibrato parameters. The vibrato rate may also be altered according to a new vibrato rate given in the score. After new time-scaled spectrum information is calculated, the program is able to synthesize the output sound with new pitch, duration and dynamic. By repeating the same procedure for each note, the program will eventually generate a piece of music with a sequence of vibrato tones.

To avoid reading the same files repeatedly, the program stores the analysis file data and the names of files that have already been read so that the program can quickly access the data.

Details of each synthesis procedure are simultaneously written to a file with the extension “.list”. This text file is generated simultaneously with the sound file. Any error that occurs will be reported in the list file.

3.1 Harmonic Analysis

There are two spectral analysis methods available in the SNDAN package [7]: the harmonic frequency McAulay-Quatieri (MQHF) method and the phase vocoder (PV) method. Both of the methods are based on the short-time Fourier transform. Although they use different analysis methods, they both store amplitude and frequency data for each harmonic at each frame. Figure 2 is a 3D plot of a violin vibrato tone's time-varying amplitude spectrum at pitch A4 (440 Hz).

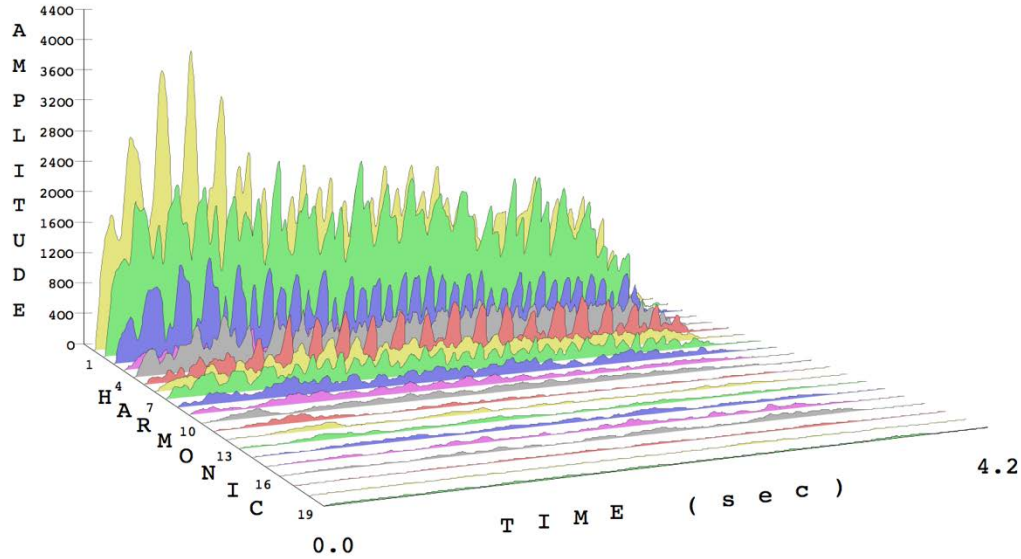


Figure 2: 3D spectrum of a violin vibrato tone at pitch A4 (440 Hz)

Analysis files contain some basic harmonic information of each tone. Table 3 lists the variables stored in an analysis file header:

Table 3: Variables in analysis file header

Variable	Type	Meaning
tl	float	Duration of the note
npts	int	Total number of time frames
dt	float	Duration of each frame
fa	float	Fundamental frequency
nhar	int	Number of harmonics
nhar1	int	Number of harmonics plus 1
cmag	float array	Amplitude value for each harmonic at each frame Amplitude of harmonic k at frame i is given by $cmag[k + i \times nhar1]$
dfr	float array	Frequency deviation for each harmonic at each frame Frequency of harmonic k at frame i is given by $fa + dfr[k + i \times nhar1]$
phas0	float array	Starting phase for each harmonic

After several experiments, we discovered that using the MQHF method, as opposed to the PV method, reduces the run-time for synthesizing a output sound. This is because the MQHF analysis file's size

decreases as fundamental frequency (f_a) increases. This is due to higher pitch tones containing fewer harmonics (in order for their frequencies to remain below the Nyquist frequency), while the frame duration (dt) is fixed, so that the total number of frames is independent of f_a . However, because the PV method always requires 2 frames per fundamental period, the number of frames increases as f_a increases. Therefore, the MQHF method is preferred, especially for long scores with high pitches, because of its significantly faster computation speed. Also, as mentioned above, the MQHF method is relatively impervious to the between-harmonics cross-talk problem.

3.2 Vibrato Parameterization

Conventional time-scale modification methods can successfully shorten or elongate a non-vibrato tone without modifying the pitch or timbral quality. However, when these methods are used on vibrato tones, their vibrato rates will also change proportionally to the time-scale. Therefore, a parametric model for vibrato tone is necessary for retaining the vibrato waveform and other timbral related parameters.

3.2.1 Frequency and amplitude spectrum

From observing 2D frequency-vs-time plots of vibrato tones, it seems reasonable to model any vibrato tone with sine waves, both for frequency and amplitude. Figure 3 is a 2D frequency spectrum plot for a violin vibrato tone with fundamental frequency at 440 Hz. The brightness of the color indicates the relative amplitude value.

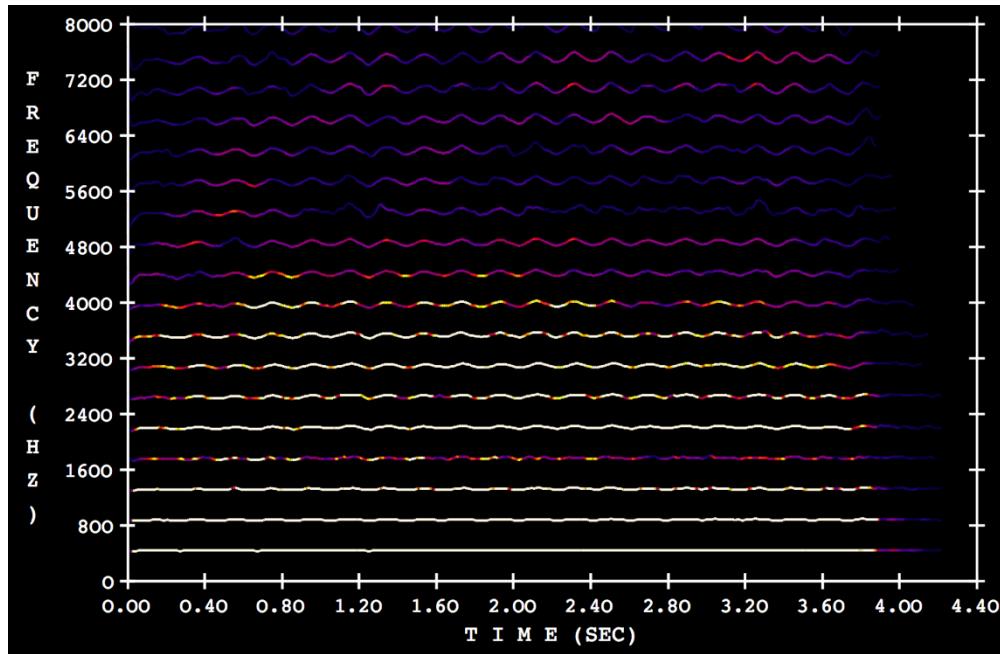


Figure 3: Frequency spectrum of a violin vibrato tone at pitch A4(440Hz)

As we can see from Fig. 3, all harmonic frequency variations can be approximated by simple sine waves. Each harmonic has the same vibrato rate. We might not expect the amplitude for each harmonic to vary at the same rate; however, from Fig. 2, the 3D plot of the violin vibrato tone, whose vertical scale is the amplitude, we can see that harmonic amplitudes vary in a periodic pattern. We refer to the amplitude

variation as *tremolo*. Figure 4 shows the amplitude tremolo of the 2nd harmonic of a 440 Hz (A₄) violin vibrato tone:

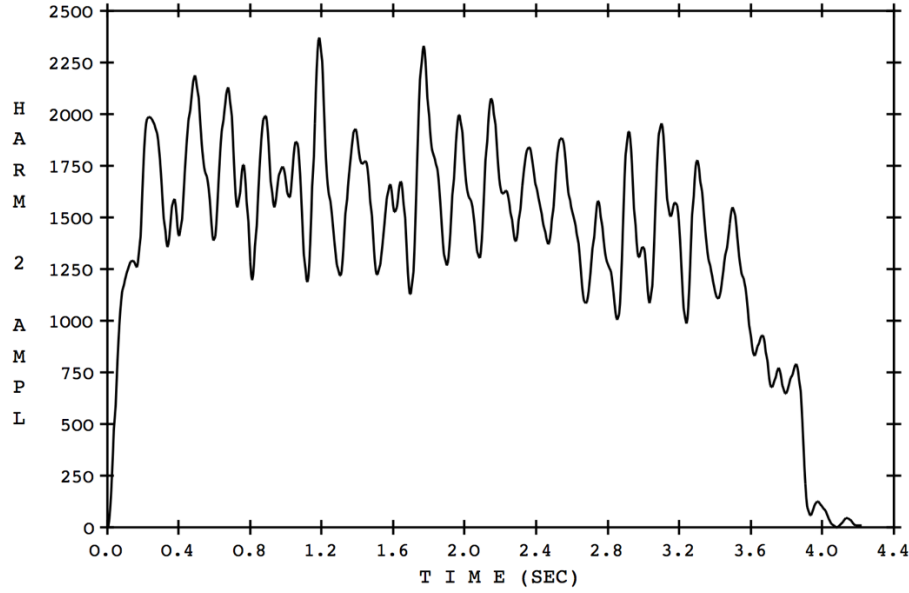


Figure 4: Amplitude tremolo of the 2nd harmonic of a violin vibrato tone at pitch A4 (440 Hz)

Different from the frequency waveform, the amplitude for each harmonic actually has two parts (attack and decay) where the amplitude increases from the sound's beginning to a decibel average (attack) or decreases from this average to the sound's end (decay). Except for these two parts, the waveform can be approximated by an appropriately modulated sine wave at the desired vibrato frequency for each harmonic.

3.2.2 Attack and decay

Since the attack and decay parts of the harmonic amplitudes can not be modeled as tremolo, and harmonic frequencies are usually steady during the attack and decay, the parametric model operates only on the middle portion of a vibrato tone, which excludes the attack and decay.

In order to locate the end of the attack and the beginning of the decay for each harmonic, the harmonic amplitudes are first converted to a decibel scale using Eq. (3.1):

$$A_{k,dB}(t) = 20 \times \log_{10} A_k(t) \quad (3.1)$$

We define the attack as the time interval that starts from the 0th frame and ends at the frame where the amplitude first reaches the average decibel value. The decay starts from the frame where the amplitude last reaches the average and ends at the final frame ($i = npts - 1$). For example, Fig. 5 shows the attack and decay of a non-vibrato tone on one of its amplitude envelopes. The blue part is the attack and the green part is the decay.

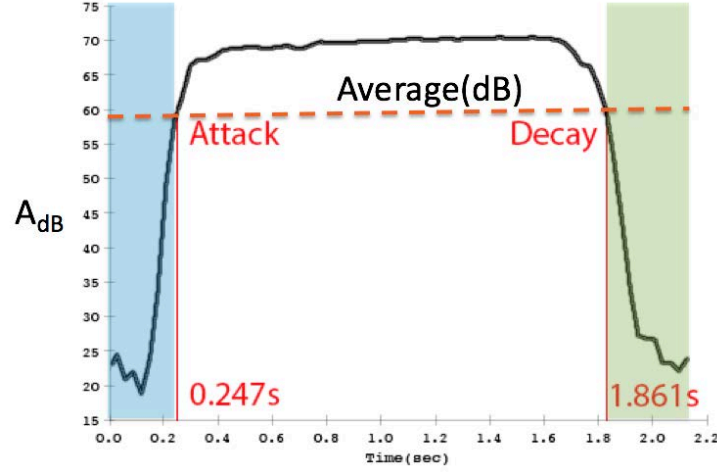


Figure 5: Attack and decay

Note that the middle section, where the vibrato parametric model operates, excludes the attack and decay parts. Then, the parametric model time-scales this middle section to a new duration. Finally, the attack and decay are concatenated back with the middle part. An exception to this strategy is necessary when the required duration is shorter than the attack and decay. This is discussed in [section 3.3.2](#).

3.2.3 Parametric model

Equations (3.2) and (3.3) show the parametric model for any vibrato tone:

$$s(t) = \sum_{k=1}^K A_k(t) \cos \left[2\pi k \left(\int \left(f_a + f_{d,k}(t) + \Delta f_v(t) \cos \left\{ 2\pi \int f_{v,k}(t) dt + \theta_{0k} \right\} \right) dt + \theta_{0k} \right) \right] \quad (3.2)$$

$$\text{where } A_k(t) = A_{m,k}(t) + \Delta A_k(t) \cos \left\{ 2\pi \int f_{v,k}(t) dt + \phi_{0k} \right\} \quad (3.3)$$

Table 4 lists the meaning corresponding to each term in the expression.

Table 4: Variables in parametric model

Term	Meaning
$s(t)$	vibrato tone signal
K	total number of harmonics
$A_k(t)$	harmonic k time-varying amplitude
f_a	average fundamental frequency in Hz
$f_{d,k}(t)$	harmonic k time-varying frequency drift
$\Delta f_v(t)$	harmonic k time-varying frequency depth (vibrato)
$f_{v,k}(t)$	time-varying vibrato rate
θ_{0k}	harmonic k starting phase for frequency
$A_{m,k}(t)$	harmonic k time-varying mean amplitude
$\Delta A_k(t)$	harmonic k time-varying amplitude depth (tremolo)
ϕ_{0k}	Harmonic k starting phase for amplitude

By observing the equations, we can see that the expressions for modeling each harmonic's amplitude and frequency are essentially the same. There are 3 parameters for both the amplitude and frequency models for each harmonic: time-varying mean (amplitude) or drift (frequency), time-varying vibrato rate, and time-varying depth. The average is straightforward: It is equal to the time average of a harmonic's vibrato or tremolo waveform. The time-varying drift measures how much the mean value is different than the average. The time-varying vibrato rate indicates how many time the vibrato waveform oscillates in 1 second. The time-varying depth measures the amplitude of the vibrato waveform.

The time-varying vibrato rate and the time-varying depth can be calculated using the heterodyne method (see [section 3.2.5](#)) after calculating the estimated average vibrato rate (see [section 3.2.4](#)). The average is equal to the average of the vibrato waveform. The time-varying drift is calculated by averaging the local maxima and minima of the waveform. Next the mean values are calculated, and then linear interpolation is used to fill out all of the frames' values.

3.2.4 Estimated vibrato rate

Determining the vibrato-dependent parameters requires first of all an estimation of the average vibrato rate. This is found by computing the Fourier transform of the frequency waveform. For example, the magnitude spectrum of of a violin's frequency waveform is shown in Fig. 6. The estimated vibrato rate corresponds to the frequency of the maximum value, about 5 Hz.

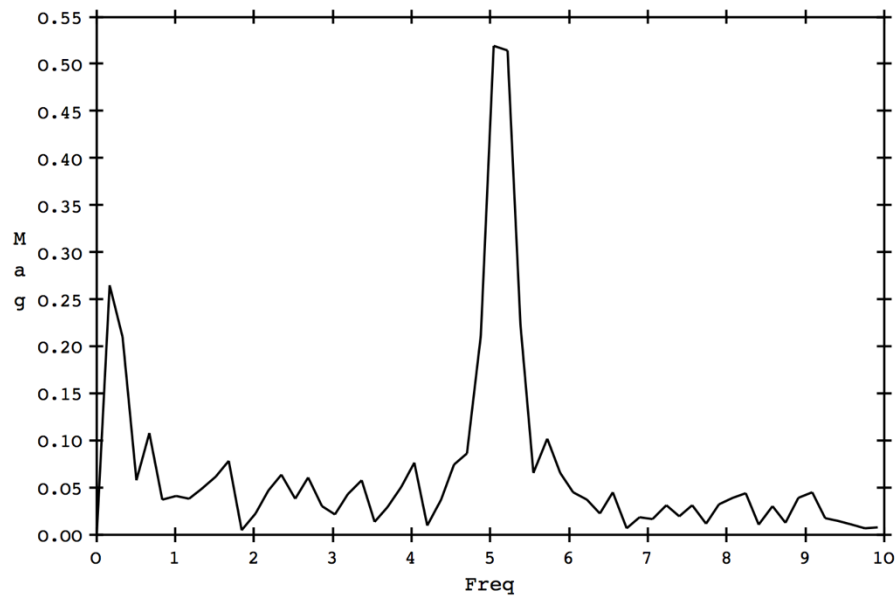


Figure 6: FFT of a vibrato tone's frequency waveform

3.2.5 Heterodyne method ^[8]

With the estimated vibrato rate f_r , the program can then use a heterodyne method to find two other parameters: time-varying vibrato rate and time-varying depth. Figure 7 shows the flow chart of the heterodyne method.

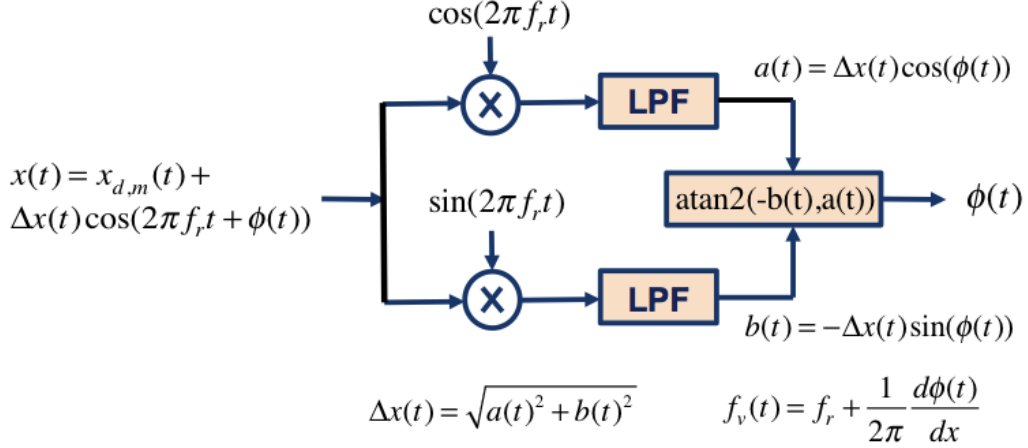


Figure 7: Flow chart of heterodyne method

Note that $x(t)$, $x_{d,m}(t)$, and $\Delta x(t)$ are actually harmonics and strictly speaking should be indexed by k . But the k index for the formulas in this section is omitted for the sake of simplicity.

Each harmonic's amplitude or frequency waveform can be approximated by equation (3.4)

$$\Delta x(t) \cos(2\pi f_r t + \phi(t)), \quad (3.4)$$

where $\Delta x(t)$ is called the depth of the waveform. It can be either the depth of the frequency waveform or the amplitude waveform. f_r is the estimated vibrato rate calculated in [section 3.2.4](#). By using the heterodyne method, the program can determine the depth $\Delta x(t)$ and the time-varying phase $\phi(t)$. The time-varying vibrato rate can be derived from the time-varying phase.

First, multiply the original signal with $\cos(2\pi f_r t)$ and $\sin(2\pi f_r t)$. We named the results as part a' and part b' :

$$a'(t) = \Delta x(t) \cos(2\pi f_r t + \phi(t)) \cos(2\pi f_r t) \quad (3.4)$$

$$b'(t) = \Delta x(t) \cos(2\pi f_r t + \phi(t)) \sin(2\pi f_r t) \quad (3.5)$$

By using the trigonometric product identities, we can simplify equations (3.4) and (3.5) to the following equations:

$$a'(t) = 0.5\Delta x(t) [\cos(4\pi f_r t + \phi(t)) + \cos(\phi(t))] \quad (3.6)$$

$$b'(t) = 0.5\Delta x(t) [\sin(4\pi f_r t + \phi(t)) - \sin(\phi(t))] \quad (3.7)$$

Passing a' and b' through low-pass filters, we obtain their low frequency parts a and b as:

$$a(t) = \Delta x(t) \cos(\phi(t)) \quad (3.8)$$

$$b(t) = -\Delta x(t) \sin(\phi(t)) \quad (3.9)$$

By taking the arctangent of the ratio of a and b , we can find the time-varying phase:

$$\phi(t) = \text{atan}\left(\frac{-b}{a}\right) = \text{atan}\left(\frac{\Delta x(t) \sin(\phi(t))}{\Delta x(t) \cos(\phi(t))}\right) = \text{atan}\left(\frac{\sin(\phi(t))}{\cos(\phi(t))}\right) \quad (3.10)$$

The time-varying vibrato rate can then be calculated using equation (3.11).

$$f_v(t) = \frac{d}{dt}\left(\frac{\phi(t)}{2\pi}\right) + f_r \quad (3.11)$$

The time-varying depth can be calculated using the results of equation (3.8) and (3.9).

$$\Delta x(t) = \sqrt{a^2 + b^2} \quad (3.12)$$

All the parameters needed have now been calculated for the parametric model.

3.3 Time-scale Modifications

Based on the parametric model, time-scale modification for vibrato tones occurs in order to produce a specified vibrato rate while stretching or compressing other parameters in order to obtain a new duration.

3.3.1 Most cases

For most cases, programmed durations will be longer than the attack and decay times combined. Therefore, the program will first perform time-scale modification on the middle part of the tone and then will concatenate the result back with the original attack and decay. The new duration of the middle part can be calculated using equation (3.13), where $T_{middle,new}$ is the new middle part duration, T_{new} is the duration of the entire new tone, T_{attack} is the attack duration, and T_{decay} is the decay duration.

$$T_{middle,new} = T_{new} - T_{attack} - T_{decay} \quad (3.13)$$

Then the program will compress or stretch the time-varying vibrato rate, time-varying depth and the time-varying drift into $T_{middle,new}$ using linear interpolation. The vibrato program is also able to synthesize vibrato tones with its vibrato rate specified by a score file. In order to accomplish this goal, the time-varying incremental phase is calculated from the new time-varying vibrato rate. Equation (3.14) shows how the phase is derived, where f_r is the estimated average vibrato rate and $f_v(t)$ is the time-varying vibrato rate calculated using equation (3.11).

$$\phi(t) = \int 2\pi(f_v(t) - f_r) dt \quad (3.14)$$

Equation (3.15) shows the final equation used for synthesizing the frequency and amplitude for each harmonic. $x_k(t)$ can be either amplitude or frequency for harmonic k . $x_{d,k}(t)$ is the time-varying drift of

frequency or amplitude, $\Delta x_k(t)$ is the time-varying depth of frequency or amplitude. $f_{r,new}$ is the new vibrato frequency specified by the score file, and $\phi(t)$ is the time-varying incremental phase calculated using equation (3.14).

$$x_k(t) = x_{d,k}(t) + \Delta x_k(t) \cos\{2\pi f_{r,new}t + \phi(t)\} \quad (3.15)$$

3.3.2 A Special case

A special case occurs when the new duration is shorter than the attack and decay combined. In this situation, the synthesized tone only consists of the attack and decay. It has no middle part.

While in the usual case the attack and decay are distinguished based on each harmonic, so that the attack ending and decay beginning frames for harmonic k are generally different those from harmonic $k+1$, in this special case, the program determines the attack and decay frames based on the rms amplitude as defined in equation (3.16). Note that $A_k(t)$ means the harmonic k amplitude value.

$$Arms(t) = \sqrt{\sum_{k=1}^K A_k(t)^2} \quad (3.16)$$

Also in this case, the synthesis of each harmonic frequency is no longer taken independently of other harmonics. All harmonic frequencies are harmonic number k times the time-varying fundamental frequency (harmonic 1), which is taken as $f_a + dfr_{wa}(t)$, where the latter, called the weighted-average frequency deviation, is given by equation (3.17).

$$dfr_{wa}(t) = \frac{\sum_{k=1}^5 \frac{1}{k} (f_k(t) - k f_a) A_k(t)}{\sum_{k=1}^5 A_k(t)} \quad (3.17)$$

Recall that the attack and decay are determined by the amplitude. There is no discontinuity of the harmonic amplitudes, but there can be discontinuities in the harmonic frequencies since the frequency at the end of the attack is not guaranteed to be equal to the frequency of the start of the decay. Therefore, the two values need to be equalized in order to avoid a sudden jump in the frequency waveform. An offset value is used to remove such a discontinuity. The offset is calculated by using the end frequency of the attack minus the start frequency of the decay. Then that offset is added to the decay frequency for every frame.

On the other hand, the amplitude waveforms still need to be independently derived for each harmonic. This is necessary for preserving the timbral quality.

3.4 Additive Synthesis

Additive synthesis is a widely used synthesis method for computer music. Originating from Fourier theory, the additive synthesis method is based on the idea that any sound signal can be expressed as a sum of sinusoids.^[9]

In the author's program, only harmonic sine waves are added. Therefore, the necessary variables for synthesis are the time-varying frequency and amplitude spectra. In order to simplify the computation processes in the parameterization part, the frequency ($f_k[i]$) spectrum in discrete time is actually stored as a fixed fundamental frequency f_a and a frequency deviation ($dfr[i]$), where i = frame number. These spectra are stored in two 1D arrays. Equation 3.18 shows how to convert the actual frequency at harmonic k to 1D array. $nhar1$ means the number of harmonics plus 1. Variables here are the same as in Table 3.

$$dfr[k + i \times nhar1] = f_k[i] - kf_a \quad (3.18)$$

Similarly, the magnitude (amplitude) spectrum can also be stored in an 1D array named $camg[i]$.

$$cmag[k + i \times nhar1] = mag_k[i] \quad (3.19)$$

With the new frequency and magnitude spectrum data, the program is able to synthesize the final output sound.

4. Conclusion

The author has successfully built a vibrato synthesis program in Music 4C package, based on the time-varying spectrum analysis of musical sounds. The program is able to synthesize vibrato music, especially bowed-string instruments' vibrato, according to a score file while preserving the timbral quality of the sample files. The success of the program demonstrates that the parametric model for vibrato tone is suitable. The parametric model can be considered as a new time-scale modification method for vibrato tones.

Synthesized output was compared with original violin tones that served as input. Although some difference was easily heard between the two, they seemed similar enough to be recognized as played by the same instrument. However, after careful listening, it is suspected that a possible reason for losing some of the violin's timbral character is due to limitations of the analysis and additive synthesis methods. The entire program only operates on harmonic frequencies. Frequencies other than harmonics are considered noise. Although harmonic information is usually enough for preserving timbral quality, for some instruments noise is a strong timbral characteristic. For bowed-string instruments, noise is not significant enough for distinguishing the instrument. Therefore, it is the author's opinion that the analysis method, the parametric model, and the additive synthesis method are adequate for this bowed-string instrument vibrato tone synthesis program.

References

- [1] J. Sundberg, “Acoustic and psychoacoustic aspects of vocal vibrato,” *STL-QPSR*, vol. 35, pp. 045–068, 1994.
- [2] J. W. Beauchamp, “Music 4C, a multi-voiced synthesis program with instruments defined in C,” *Acoustical Society of America*, vol. 113, no. 4, Mar. 2003.
- [3] J. Beauchamp, “Music 4C Introduction.”, Computer Music Project, School of Music, University of Illinois at Urbana-Champaign.
http://cmp.music.illinois.edu/beaucham/software/m4c/M4C_introHTML/M4C_intro.html
- [4] J. W. Beauchamp, D. L. Code, K.-ping Chen, G. Murakami, and G. Johnston, “Notepro, a Music Transcription Tool.”, Computer Music Project, School of Music, University of Illinois at Urbana-Champaign.
- [5] R. Maher and J. Beauchamp, “An Investigation of Vocal Vibrato for Synthesis,” *Applied Acoustics*, vol. 30, pp. 219–245, 1990.
- [6] J. Driedger and M. Meinard, “A Review of Time-Scale Modification of Music Signals,” *Applied Sciences*, vol. 6, no. 57, 2016.
- [7] J. W. Beauchamp, “Unix Workstation Software for Analysis, Graphics, Modification and Synthesis of Musical Sounds,” *Audio Engineering Society*, Preprint, 3479, C1-7, 1993.
- [8] J. W. Beauchamp, “Electronic Instrument for Synthesis, Control, and Analysis of Harmonic Musical Tone.” Ph.D. thesis, University of Illinois at Urbana-Champaign, Urbana, IL, Chapter 5, 1965.
- [9] *Additive Synthesis (Early Sinusoidal Modeling)*. Available:
https://ccrma.stanford.edu/~jos/sasp/Additive_Synthesis_Early_Sinusoidal.html.